

선물

아마루는 가게에서 선물을 사고 있다. 선물은 N 가지 종류가 있다. 각 종류의 선물의 개수는 무한대이다.

각 종류의 선물은 정해진 가격이 있다. 즉, 종류 i ($0 \leq i < N$) 선물의 가격은 양의 정수인 코인 개수 $P[i]$ 로 주어진다.

아마루는 선물 종류별 가격이 모두 다르며 감소하는 순서로 주어졌다는 것을 알고 있다. 즉, $P[0] > P[1] > \dots > P[N-1] > 0$ 이 성립한다. 추가로, 아마루는 $P[0]$ 의 값을 알고 있다. 불행하게도 아마루는 다른 가격에 대해서는 아무것도 모른다.

몇개의 선물을 사기 위해 아마루는 상점과 여러 차례의 트랜잭션을 진행한다.

각 트랜잭션은 아래의 과정으로 진행된다:

1. 아마루가 상점에 양의 정수 개수의 코인을 전달한다.
2. 상점은 뒷방 테이블에 한 더미로 코인을 둔다. 아마루는 뒷방을 전혀 볼 수 없다.
3. 상점은 각 선물을 종류 $0, 1, \dots, N-1$ 의 순서로 하나씩 고려한다. 각 선물은 각 트랜잭션에서 **정확히 한번** 고려된다.
 - 종류 i 선물을 고려할 때, 더미에 있는 코인의 개수가 $P[i]$ 이상이면,
 - 상점은 더미에서 $P[i]$ 개의 코인을 제거하고,
 - 종류 i 인 선물 하나를 테이블에 올린다.
4. 상점은 테이블에 남아 있는 모든 동전과 모든 선물을 아마루에게 전달한다.

각 트랜잭션이 시작하기 직전에 테이블 위에는 코인이나 선물이 하나도 없다는 것에 주의하라.

당신이 할 일은 아마루에게 몇차례의 트랜잭션을 하도록 지시하는 것이다. 그 목표는 다음과 같다.

- 각 트랜잭션에서 **최소 하나의 선물**을 산다.
- 전체적으로 아마루는 종류 i 선물을 **정확히 i 개** 산다 ($0 \leq i < N$). 규칙에 따라 종류 0 선물은 사지 않아야 된다는 것에 주의하라.

아마루가 트랜잭션의 개수를 최소화할 필요는 없다. 아마루가 가진 코인은 무한히 많다.

Implementation Details

다음 함수를 구현해야 한다.

```
void buy_souvenirs(int N, long long P0)
```

- N : 선물의 종류 개수.

- P_0 : $P[0]$ 의 값.
- 이 함수는 각 테스트 케이스에 대해 정확히 한번 호출된다.

위 함수는 아래 함수를 호출해서 아마루에게 트랜잭션을 진행하도록 지시할 수 있다.

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : 아마루가 상점에 건넨 코인 개수.
- 함수는 pair를 리턴한다. Pair의 첫번째 원소는 배열 L 이다. 이 배열에는 사게 된 선물의 종류 번호들이 증가하는 순서로 저장되어 있다. Pair의 두번째 원소는 정수 R 이다. 이 값은 트랜잭션이 끝난 후 되받은 코인의 개수이다.
- $P[0] > M \geq P[N - 1]$ 이 반드시 성립해야 한다. 조건 $P[0] > M$ 은 아마루가 종류 0 선물을 사지 않게 된다는 것을 보장하고, 조건 $M \geq P[N - 1]$ 은 아마루가 최소 1개의 선물을 사게 된다는 것을 보장한다. 이 조건이 만족되지 않으면 제출은 Output isn't correct: Invalid argument의 결과를 받는다. $P[N - 1]$ 의 값은 입력으로 주어지지 않는다는 것에 주의하라. $P[0]$ 의 값은 주어진다.
- 각 테스트 케이스에 대해 이 함수는 최대 5000번 호출할 수 있다.

그레이더는 **적용적이지 않다**. 즉, P 배열의 값은 초기에 정해져 있다.

Constraints

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ ($0 \leq i < N$).
- $P[i] > P[i + 1]$ ($0 \leq i < N - 1$).

Subtasks

Subtask	Score	Additional Constraints
1	4	$N = 2$
2	3	$P[i] = N - i$ ($0 \leq i < N$).
3	14	$P[i] \leq P[i + 1] + 2$ ($0 \leq i < N - 1$).
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ ($0 \leq i < N - 2$), $P[i] \leq 2 \cdot P[i + 1]$ ($0 \leq i < N - 1$).
6	33	추가적인 제한이 없다.

Example

다음 호출을 보자.

```
buy_souvenirs(3, 4)
```

선물의 종류는 $N = 3$ 가지가 있으며 $P[0] = 4$ 이다. P 배열의 내용은 다음 3가지가 가능함을 알 수 있다: $[4, 3, 2]$, $[4, 3, 1]$, $[4, 2, 1]$.

`buy_souvenirs` 함수가 `transaction(2)`를 호출했고 그 리턴 값은 $([2], 1)$ 이었다고 하자. 리턴 값의 의미는 아마루가 종류 2 선물을 하나 샀고 상점은 코인 1개를 되돌려 주었다는 뜻이다. 이제, 아마루는 $P = [4, 3, 1]$ 임을 알 수 있다. 아래가 성립하기 때문이다.

- 만약 $P = [4, 3, 2]$ 이라면, `transaction(2)`은 $([2], 0)$ 을 리턴해야 한다.
- 만약 $P = [4, 2, 1]$ 이라면, `transaction(2)`은 $([1], 0)$ 을 리턴해야 한다.

이제 `buy_souvenirs`는 `transaction(3)`을 호출할 수 있다. 이 호출은 $([1], 0)$ 을 리턴할 것이다. 즉, 이 호출로 아마루는 종류 1 선물을 하나 샀고 상점은 0개의 코인을 되돌려 주었다. 지금까지 아마루는 종류 1과 종류 2 선물을 한개씩 샀다.

마지막으로 `buy_souvenirs`는 `transaction(1)`을 호출할 수 있고 호출은 $([2], 0)$ 을 리턴한다. 즉, 아마루는 방금 호출로 종류 2 선물을 하나 더 샀다. `transaction(2)`를 사용해도 동일한 선물을 살 수 있다. 현재 시점까지 아마루는 종류 1 선물 하나와 종류 2 선물 2개를 샀으므로 문제의 조건을 만족했다.

Sample Grader

입력 형식:

```
N
P[0]  P[1]  ...  P[N-1]
```

출력 형식:

```
Q[0]  Q[1]  ...  Q[N-1]
```

$Q[i]$ 는 전체 과정에서 종류 i 선물을 산 개수를 의미한다. ($0 \leq i < N$)